

9	8	7	6	5	4	3	2	1	0

1024 проби
10 животни $2^{10} = 1024$

1024 проби, 1 отрова
10 животни, 24 часа
да разберем като я
дадем на
животно

$$2^0 = 1 = 1$$

$$2^1 = 2 = 10$$

$$2^2 = 4 = 100$$

$$2^3 = 8 = 1000$$

$$2^4 = 16 = 10000$$

$$2^5 = 32 = 100000$$

$$2^6 = 64 = 1000000$$

$$2^7 = 128 = 10000000$$

$$2^8 = 256 = 100000000$$

$$2^9 = 512 = 1000000000$$

$$2^{10} = 1024 = 10000000000$$

$$0 \div 1023$$

$$56_{10} = ?_2$$

	$\div 2$
56	0
28	0
14	0
7	1
3	1
1	1
0	

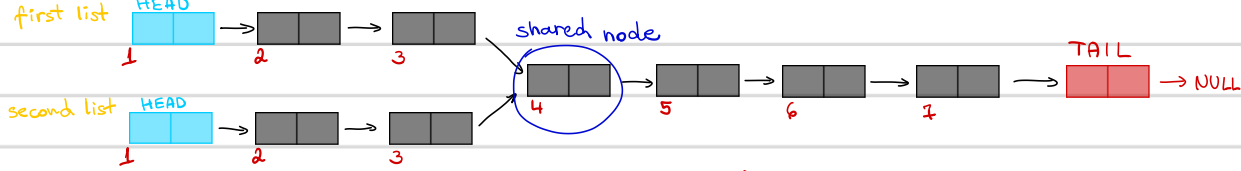
$$= 111000_2$$

Сливащи се свързани списъци

Дадени са ви указатели към първите елементи на два ациклически едносвързани списъка. Можете ли с константна памет и линейно време да определите дали в някой елемент единият списък не се "слива" във втория (тоест двата да завършват в един и същ елемент)? Ако да, можете ли да намерите кой е първият им общ елемент? Опишете алгоритъма си.

Линка на края
(за свързване на следващия)
⇒ we have tail and head

Linked Lists



↳ in constant memory
↳ in linear time complexity

Алгоритъм:

1. Трябва да разберем дали списъците са свързани, като ги обходим и 2-та, за да проверим последният им елемент (tail) дали е еднакъв.
2. Ако са свързани, следва да разберем, къде точно се сливат? - На всяка стъпка да проверяваме за еднаквост на елементите и ще ни доведе до $O(n^2)$ (квадратна сложност). В тази насока, ако погледнем от друг ъгъл на нещата, нас какво ни интересува? - къде точно се сливат, т.е. можем да си изберем подходящ елемент от 2-та списъка, от който да започнем да разглеждаме, къде се свързват. А кой е подходящият елемент?
Взимаме по-дългия списък, като него разглеждаме от абсолютната стойност на разликата или $abs(listLength2 - listLength1)$
3. Итерираме всеки елемент докато не открием обичия.

Сложност: time complexity $O(n+m)$ space complexity $O(1)$

Нагоре-надолу

По случаен начин е избрано цяло число num между 1 и 1000. Имате функция $guess(x)$, която връща 0, ако $num == x$, -1 ако $x < num$, и +1 ако $x > num$. Изградете алгоритъм, който, познава числото num с минимален брой викания на функцията $guess(x)$. Определете броя извиквания при най-лошия избор на num за вашия алгоритъм.

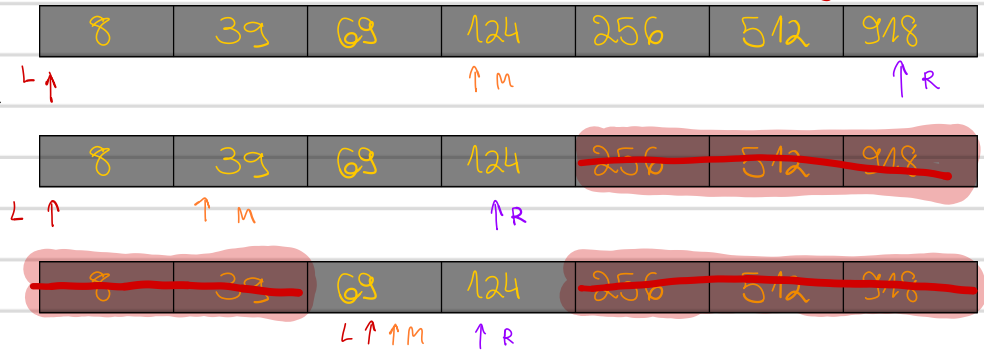
binary search $O(\log_{2} n)$
range 1 ÷ 1000
Number to look for : 69

EDGE CASES :

- резултата сортирана ли е ?
- Гарантирано ли е че числото ще бъде в резултата ?

Алгоритъм:

1. Избираме си пивот/ mid (необязателно елемент) $(start + end) / 2$
2. Използваме ф-ята $guess$, за да ни покаже дали сме намерили елемента, или не. Ако не сме, да знаем в кой друг диапазон, най-вероятно е, по-малкия или по-големия, сравнен с нашия пивот/ mid . Така повтаряме действията, докато не го открием, или не свършат елементите.



```
int left = 1, right = 1000;
while (left <= right) {
    int mid = (left + right) / 2;
    int res = guess(mid);
    if (res == 0)
        return mid;
    if (res < 0)
        right = mid - 1;
    else
        left = mid + 1;
}
```

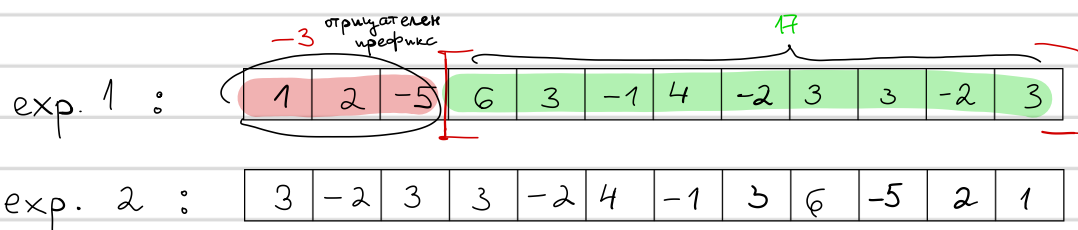
Сложност : time complexity $O(\log_2 n)$
space complexity $O(1)$

програмна реализация

Под-интервал с максимална сума

Даден е масив с N числа. Изградете алгоритъм, който намира негов подинтервал с максимална сума. Под-интервал е поредица от последователни негови елементи

Очевидно решение е да проверим всеки подинтервал на масива и да изведем този с най-голяма сума, обаче тогава сложността по време ще е $O(n^2)$, а ние търсим **оптимално решение**.
Един интервал е потенциално оптимален, ако няма отрицателен префикс **- или** суфикс **ниг**



Ще използваме greedy алгоритъм, който да избира така интервала, че да няма отрицателен префикс, и да тества всички възможни суфикси, така че дори да